



UNIVERSIDAD TECNOLÓGICA
FACULTAD DE INGENIERÍA EN SISTEMAS
CAMPUS VICTOR LEVIS SASSO
INGENIERÍA WEB



Fecha Límite de Entrega:
Del 14 de noviembre de 2025

II SEMESTRE 2025

Instructor: Ing. Irina Fong

Grupos: ISF131 / ISF132

 Guía de Laboratorio Práctico: “CRUD - API Fetch y MySQL –
(Guardar, Editar, Buscar)”

Este laboratorio lo pueden hacer en grupo de dos estudiantes. Con la finalidad de que ambos colaboren.

**Convertir una
matriz
asociativa PHP a
JSON**

Las matrices asociativas son matrices que utilizan **claves con nombre** como índices para sus valores.

```
$book = array("id"=>101, "label"=>"Lorem Ipsum");  
echo json_encode($book);
```

Ejemplo:

```
// $this->controlError[$field]  
$mensaje = "Producto no creado";  
$response = [ "success" => false,  
              "message" => $mensaje,  
              "accion" => "Guardar"];  
  
}  
  
header("Content-Type: application/json"); // Asegúrate de enviar JSON  
echo json_encode($response);  
exit;  
break;
```



UNIVERSIDAD TECNOLÓGICA
FACULTAD DE INGENIERÍA EN SISTEMAS
CAMPUS VICTOR LEVIS SASSO
INGENIERÍA WEB



A. Título de la experiencia

CRUD con Fetch + PHP OOP + MySQL

(Guardar, Editar y Buscar productos a través de un formulario dinámico)

API Fetch	API Fetch ofrece una forma sencilla y moderna de realizar solicitudes HTTP en JavaScript mediante promesas . Reemplaza métodos antiguos como XMLHttpRequest y permite una gestión eficiente de la recuperación de datos, el análisis de respuestas y la gestión de errores. Tanto si se usa <code>`.then()`</code> como <code>`async/await`</code> , Fetch facilita y agiliza el trabajo con las API . Gracias a las comprobaciones de estado y la gestión de errores adecuadas , ayuda a crear aplicaciones web fiables e intuitivas .
------------------	--

B. Temas

- Formulario HTML y envío asíncrono con **fetch**
- FormData y formato **multipart/form-data**
- Programación orientada a objetos en PHP
- Conexión segura a MySQL usando PDO
- Estructura de control **switch** en PHP y JavaScript
- Validación de entradas y manejo de errores
- Alertas amigables con **SweetAlert 2**

C. Objetivos

Tipo Descripción

Desarrollar un formulario web que permita **guardar, editar y buscar** productos mediante peticiones **fetch** y operaciones CRUD en MySQL, aplicando buenas prácticas de PHP OOP.



UNIVERSIDAD TECNOLÓGICA
FACULTAD DE INGENIERÍA EN SISTEMAS
CAMPUS VICTOR LEVIS SASSO
INGENIERÍA WEB



Objetivos Específicos:

- Crear clases de **conexión** y **producto** en PHP
- Centralizar las acciones “Guardar” y “Modificar” en un solo archivo utilizando **switch**.
- Mostrar mensajes de éxito / error con **SweetAlert 2**.
- Utilizar Bootstrap.

D. Metodología

1. **Mini-explicación** (docente) sobre fetch, FormData y flujo petición-respuesta en PHP.
2. **Construcción guiada:**
 1. Maquetar formulario HTML.
 2. Programar **script.js** para capturar eventos y llamar a fetch.
 3. Reutilizar el **Modelo/conexion.php** (clase DB), crear el **Modelo/Productos.php** (clase Producto).
 4. Desarrollar **registrar.php** con switch (\$accion) que reciba \$_POST.
3. **Pruebas** de guardar, editar y buscar.
4. **Socialización y retroalimentación.**

E. Instrucciones paso a paso

1. **BD:** crear productosdb y la tabla

sql

CopiarEditar

CREATE TABLE productos (

id INT AUTO_INCREMENT PRIMARY KEY,

codigo VARCHAR(20) NOT NULL,

producto VARCHAR(100) NOT NULL,



UNIVERSIDAD TECNOLÓGICA
FACULTAD DE INGENIERÍA EN SISTEMAS
CAMPUS VICTOR LEVIS SASSO
INGENIERÍA WEB



precio DECIMAL(10,2) NOT NULL,

cantidad INT NOT NULL

); // si no agrega algún campo se le descontará 5 puntos.

2. Archivos a crear

Carpeta/archivo	Descripción breve
Modelo/conexion.php	Clase DB (singleton o simple) que expone insertSeguro, updateSeguro, query.
Modelo/Productos.php	Clase Producto con propiedades y métodos guardar(), editar(), buscar(), validación.
registrar.php	Recibe \$_POST, ejecuta switch(\$_POST['Accion']) → Guardar ó Modificar, retorna JSON success / message / accion / errors.
index.html	Formulario con inputs: Código, Producto, Precio, Cantidad + botón <i>Registrar / Actualizar</i> .
script.js	Captura clic, crea FormData, envía con fetch, procesa respuesta con SweetAlert2.

3. **Validar** en PHP los campos obligatorios antes de guardar/editar.

4. **Mostrar** cada resultado con SweetAlert2 y recargar la tabla (función ListarProductos()).

F. Recursos

- Video guía (referencial): <https://youtu.be/AXZGTOd8ASk>
- SweetAlert2 CDN:

html

CopiarEditar

```
<script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script>
```

- Servidor local (WampServer / XAMPP).



UNIVERSIDAD TECNOLÓGICA
FACULTAD DE INGENIERÍA EN SISTEMAS
CAMPUS VICTOR LEVIS SASSO
INGENIERÍA WEB



- Editor de código (VS Code).

El CDN de bootstrap

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">

    <title>CRUD con PHP y ... Fetch</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/css/
bootstrap.min.css" rel="stylesheet" integrity="
sha384-rbsA2VBKQhggwzxH7pPCaAqO46MgnOM80zW1RWuH61DGLwZJEdK2Kadq2F9CUG65"
crossorigin="anonymous">

  </head>
```



UNIVERSIDAD TECNOLÓGICA
FACULTAD DE INGENIERÍA EN SISTEMAS
CAMPUS VICTOR LEVIS SASSO
INGENIERÍA WEB



Fecha Límite de Entrega:
Del 14 de noviembre de 2025

II SEMESTRE 2025

Instructor: Ing. Irina Fong

Grupo: **ISF131 / ISF132**

Estudiante: _____

Fecha de Entrega: _____

 **Guía de Laboratorio Práctico: “CRUD - API Fetch y MySQL –
(Guardar, Editar, Buscar)”**

G. Rúbrica de evaluación (100 pts)

- Debe subir el repositorio con el proyecto. Valor 15 puntos. Si al momento de subir la nota no se encuentra el repositorio se le descontará de los puntos obtenidos.

Criterio	Puntos	Pts Obtenidos
Formulario realiza guardar, editar y buscar correctamente	10	
Uso de Bootstrap para el diseño y botones (Agregar Alert)	10	
Lógica “Guardar / Modificar” centralizada con switch	15	
Uso de switch también en JavaScript	10	
Clases DB implementadas	10	
Clases Producto implementadas	10	
Validaciones + control de errores (servidor y cliente)	10	
Respuestas JSON con success, message, errors	10	
Uso de SweetAlert 2 para éxito y errores	10	
Código limpio, comentarios y estructura de carpetas	5	
Total	100	

Notas finales



UNIVERSIDAD TECNOLÓGICA
FACULTAD DE INGENIERÍA EN SISTEMAS
CAMPUS VICTOR LEVIS SASSO
INGENIERÍA WEB



- Puede ampliarse para incluir DELETE o paginación como reto extra.
- Puntualidad estaría dentro de la Primera, equivale a 15 puntos. Se le descontará si no es puntual.
- Debe subir el repositorio con el proyecto. Valor 15 puntos. Si al momento de subir el repositorio no se encuentra la información se le descontará del los puntos obtenidos.

Anexos

¿Cómo utilizar la API de obtención de JavaScript para obtener datos?

La **API Fetch** es una forma moderna de realizar **solicitudes HTTP** en **JavaScript**. Está integrada en la mayoría de los navegadores y permite a los desarrolladores realizar solicitudes de red (como obtener datos de un servidor) de forma sencilla y eficiente. La API Fetch reemplaza técnicas antiguas como **XMLHttpRequest** y **los métodos AJAX de jQuery**.

Sintaxis

```
fetch(url)
  .then(respuesta => respuesta.json())
  .then(datos => console.log(datos))
  .catch(error => console.error(error))
```

En la sintaxis anterior:

- **url** : la URL desde la que desea obtener datos.
- **.then()** : maneja la respuesta cuando la solicitud es exitosa.
- **.catch()** : captura cualquier error si la solicitud falla.

Cómo funciona Fetch

La función `fetch()` es una forma moderna de realizar solicitudes HTTP en JavaScript mediante promesas. Funciona así:

- **fetch(url)** envía una solicitud a la URL dada (el valor predeterminado es GET).
- Devuelve una **Promesa** que se resuelve en un objeto Respuesta.

- Utilice **.then()** para procesar la respuesta (por ejemplo, `response.json()`).
- Utilice **.catch()** para manejar errores de red.
- No arroja errores para códigos de estado **HTTP** como 404: verifique `response.ok` manualmente.

La función **fetch()** envía una solicitud a la API y recupera los datos del producto 1 de la URL proporcionada.

La respuesta se analiza en JSON con `.then(response => response.json())` y los datos resultantes se registran en la consola, mientras que cualquier error se detecta y se muestra con **.catch()**.

```
async function fetchData() {
  try {
    const response = await fetch('https://fakestoreapi.com/products/1');
    if (!response.ok) {
      throw new Error('Network response was not ok');
    }
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error('Error:', error);
  }
}

fetchData();
```

Ahora entendamos esto con la ayuda de un ejemplo:

```
fetch('https://fakestoreapi.com/products/1')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error('Error:', error));
```



```
C:\Users\GFG0569>node index.js
{
  id: 1,
  title: 'Fjallraven - Foldsack No. 1 Backpack, Fits 15 Laptops',
  price: 109.95,
  description: 'Your perfect pack for everyday use and walks in the forest. Stash your yday',
  category: 'men's clothing',
  image: 'https://fakestoreapi.com/img/81fPKd-2AYL._AC_SL1500_.jpg',
  rating: { rate: 3.9, count: 120 }
}
```

Obteniendo datos de una API

- La función **fetch()** envía una solicitud a la API y recupera los datos del producto 1 de la URL proporcionada.
- La respuesta se analiza en JSON con `.then(response => response.json())` y los datos resultantes se registran en la consola, mientras que cualquier error se detecta y se muestra con `.catch()`.

Manejo del estado de respuesta HTTP

Gestionar el estado de respuesta [HTTP](#) en la API Fetch te ayuda a gestionar diferentes resultados según la respuesta del servidor, como códigos de éxito o de error. Puedes consultar el estado para determinar qué acción tomar.

```
fetch('https://fakestoreapi.com/products/1')
  .then(response => {
    if (!response.ok) {
      throw new Error('Network response was not ok');
    }
    return response.json();
  })
  .then(data => console.log(data))
  .catch(error => console.error('Error:', error));
```

Uso de async/await con Fetch API

Usar [async/await](#) con la API Fetch permite gestionar código asíncrono de forma más legible. Aquí tienes un ejemplo sencillo para obtener datos.

```
async function fetchData() {
  try {
```

```
const response = await fetch('https://fakestoreapi.com/products/1');
if (!response.ok) {
  throw new Error('Network response was not ok');
}
const data = await response.json();
console.log(data);
} catch (error) {
  console.error('Error:', error);
}
}
```

Uso de async/await con Fetch API

- **Función asíncrona:** La función `fetchData` está marcada como `async`, lo que permite usar `await` dentro de ella. Esto hace que el código asíncrono se parezca más al código síncrono, mejorando la legibilidad.
- **await fetch():** La palabra clave `await` pausa la ejecución de la función hasta que se complete la solicitud `fetch()` y se reciba la respuesta. Esto evita la necesidad de varios métodos `.then()` y facilita la lectura del código.
- **await response.json():** Una vez recibida la respuesta, se usa `await` para analizar el cuerpo de la respuesta en formato JSON. Esto también garantiza que el código espere a que se complete el análisis de JSON antes de continuar.
- **Manejo de errores:** el uso de `try/catch` garantiza que cualquier error durante la solicitud de búsqueda o al analizar la respuesta se detecte y se gestione con elegancia, lo que evita que la aplicación se bloquee.

La gestión de errores en la API Fetch garantiza la correcta gestión de problemas como fallos de red o respuestas no válidas. A continuación, se muestra un ejemplo sencillo para demostrar cómo gestionar errores con Fetch.

```
async function fetchData() {  
  try {  
    const response = await fetch('https://fakestoreapi.com/products/100');  
  
    // Check if the response was successful  
    if (!response.ok) {  
      throw new Error('Network response was not ok');  
    }  
  
    // Parse the response body to JSON  
    const data = await response.json();  
  
    // Log the data  
    console.log(data);  
  } catch (error) {  
    // Handle any errors that occurred during the fetch  
    console.error('Error:', error);  
  }  
}  
  
// Call the async function  
fetchData();
```

💡 Consejos

Siempre que uses `fetch(...).then(res => res.json())`, asegúrate de que:

- Tu archivo PHP tenga **solo echo json_encode(...)** como salida.
- No haya echo, print, var_dump, ni espacios extra **fuera de eso**.
- El header("Content-Type: application/json"); esté al inicio del archivo.

Ejemplo de método de guardar Producto de la Clase Productos.



UNIVERSIDAD TECNOLÓGICA
FACULTAD DE INGENIERÍA EN SISTEMAS
CAMPUS VICTOR LEVIS SASSO
INGENIERÍA WEB



```
public function guardarProductos(){  
    $data = array(  
        "codigo" => "$this->Codigo",  
        "producto" => "$this->Producto",  
        "precio" => $this->Precio,  
        "cantidad" => $this->Cantidad);  
  
    $this->pdo->insertSeguro("productos",$data);  
} //fin registrarProductos  
  
public function listarProductos($consulta){  
    return $this->pdo->Arreglos($consulta);  
} //listarProductos
```



UNIVERSIDAD TECNOLÓGICA
FACULTAD DE INGENIERÍA EN SISTEMAS
CAMPUS VICTOR LEVIS SASSO
INGENIERÍA WEB



```
1 Registrar.addEventListener("click",() =>{
2     //¿Qué es FormData?
3     /*FormData es una clase de JavaScript que se utiliza
4     para construir fácilmente pares clave-valor que simulan
5     los datos de un formulario HTML. Es muy útil cuando
6     quieres enviar datos al servidor usando fetch()
7     o XMLHttpRequest mediante el método POST*/
8
9     fetch("registrar.php",{
10         method:"POST",
11         body: new FormData(frm)
12     }).then(response => response.json())
13     .then(response => {
14
15
16         if (response.success) {
17             switch (response.accion) {
18                 case "Guardar":
19                 case "Modificar":
20                     const titulo = response.accion === "Guardar" ? "Registrado" : "Actualizado";
21
22                     console.log(
23                         response.accion === "Guardar"
24                             ? "Producto guardado:"
25                             : "Producto actualizado:",
26                         response.data
27                     );
28
29                     Swal.fire({
30                         icon: "success",
31                         title: titulo,
32                         showConfirmButton: false,
33                         timer: 1500
34                     });
35
36                     frm.reset();
37                     ListarProductos();
38                     break;
39
40                     default:
41                         Swal.fire({
42                             icon: "info Error ",
43                             title: "Acción realizada - Fallas  ",
44                             text: response.message
45                         });
46                 }
47
48             } else {
49                 Swal.fire({
50                     icon: "error",
51                     title: "Error",
52                     text: response.message
53                 });
54                 console.warn("Errores:", response.message);
55             }
56         }
```



**UNIVERSIDAD TECNOLÓGICA
FACULTAD DE INGENIERÍA EN SISTEMAS
CAMPUS VICTOR LEVIS SASSO
INGENIERÍA WEB**



UNIVERSIDAD TECNOLÓGICA
FACULTAD DE INGENIERÍA EN SISTEMAS
CAMPUS VICTOR LEVIS SASSO
INGENIERÍA WEB

```
1 <?php
2 include("Modelo/conexion.php");
3 $pdo = new mod_db();
4 include("Modelo/Productos.php");
5 include("Modelo/SanitizarEntrada.php");
6
7 $myProducto = new ObjProductos($pdo);
8 $myProducto->RecibirDatos($_POST);
9
10 $Accion = $_POST['Accion'];
11
12 switch ($Accion) {
13
14     case "Guardar":
15
16
17
18         $myProducto->setRequiredFields(['Codigo', 'Producto', 'Precio', 'Cantidad']);
19         $myProducto->validate();
20
21         //Si no tiene errores
22         if (!$myProducto->Errores){
23
24
25             $myProducto->RegistrarDatos();
26             $myProducto->guardarProductos();
27             $mensaje = "Producto Creado";
28
29             $response = [ "success" => true,
30                           "message" => $mensaje,
31                           "accion" => "Guardar"];
32
33
34         }else {
35
36             //$this->controlError[$field]
37             $mensaje = "Producto no creado";
38             $response = [ "success" => false,
39                           "message" => $mensaje,
40                           "accion" => "Guardar"];
41
42         }
43
44         header("Content-Type: application/json"); // Asegúrate de enviar JSON
45         echo json_encode($response);
46         exit;
47         break;
48
49
50     case "Modificar":
51
52
53
54         $myProducto->setRequiredFields(['Codigo', 'Producto', 'Precio', 'Cantidad']);
55         $myProducto->validate();
56
57         //Si no tiene errores
58         if (!$myProducto->Errores){
59
60
61             $myProducto->RegistrarDatos();
62             $myProducto->actualizarProducto();
63
64             $mensaje = "Producto Actualizado";
65             $response = [ "success" => true,
66                           "message" => $mensaje,
67                           "accion" => "Modificar"];
68             //$errors" => [],
69
70
71         }else {
72             $mensaje = "El Producto no fue actualizado";
73
74             $response = [ "success" => false,
75                           "message" => $mensaje,
76                           "accion" => "Modificar"];
77             //$errors" => [],
78
79         }//fin del else
80
81         header("Content-Type: application/json"); // Asegúrate de enviar JSON
82         echo json_encode($response);
83         exit;
84
85         break;
86         default:
87
88             $response = [ "success" => false,
89                           "message" => "El Producto no fue actualizado",
90                           "errors" => ["con errores"],
```